

NM - HW8

The probability to choose a random direction, using spherical coordinates (on a unit sphere), is given by

$$1 = \iint_{\Omega} P(\theta, \phi) = \iint_{\Omega} r^2 \sin(\theta) d\theta d\phi = [r = 1] = \left(\int_0^{\pi} \sin(\theta) d\theta \right) \left(\int_0^{2\pi} d\phi \right) = p(\theta)p(\phi) \quad (1)$$

$$1 = \int_0^{2\pi} \tilde{p}(\phi) = N \int_0^{2\pi} d\phi = 2\pi N \rightarrow p(\phi) = \frac{1}{2\pi} \quad (2)$$

$p(\phi)$ can be sampled uniformly from $[0, 2\pi]$

$$1 = \int_0^{\pi} \tilde{p}(\theta) = N \int_0^{\pi} \sin(\theta) d\theta = 2N \rightarrow p(\theta) = \frac{1}{2} \sin(\theta) \quad (3)$$

$p(\theta)$ is not distributed uniformly on $[0, \pi]$ and will be sampled inversly

$$P(\theta) = \int_0^{\theta} \frac{1}{2} \sin(\theta') d\theta' = \frac{1}{2} (1 - \cos(\theta)) \rightarrow \theta(P) = \arccos(1 - 2P) \quad (4)$$

and now we sample P uniformly from $[0, 1]$

In [1]:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def RandDir():
5     """
6     returns a random, uniformly distributed unit vector
7     e = [e1, e2, e3]
8     """
9     phi = np.random.random(1) * 2 * np.pi
10    theta = np.arccos(1 - 2 * np.random.random(1))
11
12    e = np.zeros([3])
13    e[0] = np.sin(theta) * np.cos(phi)
14    e[1] = np.sin(theta) * np.sin(phi)
15    e[2] = np.cos(theta)
16
17    return e
```

Step lengths are distributed exponentially

$$p(L) = \frac{1}{\lambda} e^{-\frac{L}{\lambda}} \longrightarrow P(L) = \int_0^L \frac{1}{\lambda} e^{-\frac{L'}{\lambda}} dL' = 1 - e^{-\frac{L}{\lambda}} \quad (5)$$

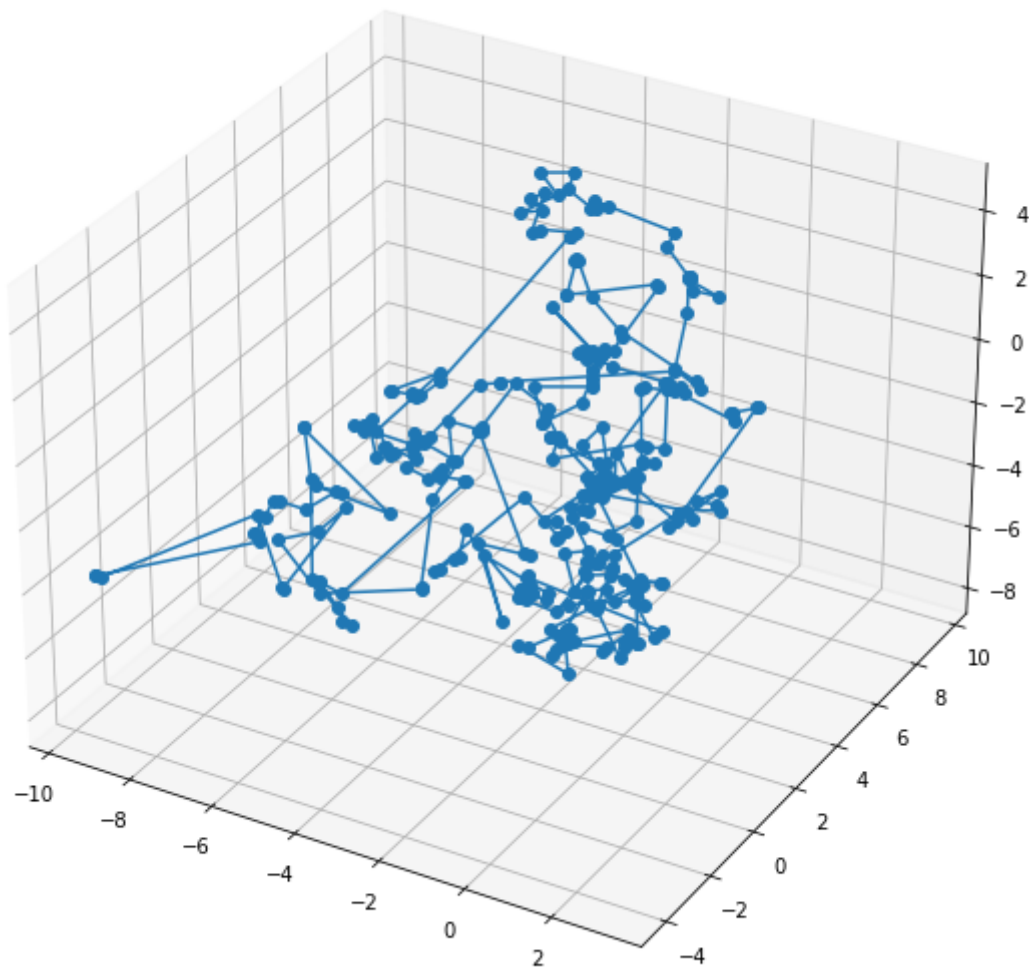
$$L(P) = \lambda \ln\left(\frac{1}{1-P}\right); P \in [0, 1] \quad (6)$$

In [2]:

```

1 def RandWalkExp3D(p, Lambda, N):
2
3     """
4     RandWalkExp3D (starting point, mean free path 'lambda', number of steps )
5     """
6
7     position = np.zeros([N,3])
8
9     ## position
10    ## (x1,y1,z1)
11    ## (x2,y2,z2)
12    ## .
13    ## .
14    ## (xN,yN,zN)
15
16    for i in range(0, N - 1):
17        L = Lambda * np.log(1 / (1 - np.random.random(1)) )
18        p = p + L * RandDir()
19        position[i + 1] = p
20    return position
21
22 p0 = [1,2,3]
23 y = RandWalkExp3D(p0 ,1, 300)
24 fig = plt.figure(figsize=[10,10])
25 ax = fig.add_subplot(111, projection='3d')
26
27 ax.plot(y[:,0],y[:,1],y[:,2], 'o-')
28 plt.show()
29
30 def RandomStep(p, Lambda):
31
32     """
33     Takes a 3D point and moves in a random direction,
34     according to exponential probability and a given mean free path.
35     Returns the 3D point after the random walk.
36     """
37
38     L = Lambda * np.log(1 / (1 - np.random.random(1)) )
39     p = p + L * RandDir()
40     return p

```



1.2

Let's define g as a function that receives a position array

In [3]:

```
1 def gcube(M):
2
3     """
4     receives Nx3 array of positions and returns N'x3
5     array of N' position that are inside g
6     """
7
8     MM = []
9     for i in range(len(M)):
10         if abs(M[i][0]) < 1 and abs(M[i][1]) < 1 and abs(M[i][2]) < 1:
11             MM.append(M[i])
12     return MM
13
14 A = [ [0.1,0.1,0.1], [0.1,2.0,0.1], [0.1,0.1,0.1], [2.0,0.1,0.1], [0.1,0.1,0.1] , [3.0,
15 gcube(A)
```

Out[3]:

```
[[0.1, 0.1, 0.1], [0.1, 0.1, 0.1], [0.1, 0.1, 0.1]]
```

1.3

In [4]:

```

1  def RandomStep(p, Lambda):
2
3      """
4      Takes a 3D point and moves in a random direction,
5      according to exponential probability and a given mean free path.
6      Returns the 3D point after the random walk.
7      """
8
9      L = Lambda * np.log(1 / (1 - np.random.random(1)) )
10     p = p + L * RandDir()
11     return p
12
13 def NeutronInit(N):
14     M1 = np.random.uniform(-1,1, [N,3])
15     return M1
16
17 def NeutronSimulate(N , g , Lambda , k, steps):
18
19     # initialization
20     M1 = NeutronInit(N)
21
22     for i in range(steps):
23
24         # random step
25         for j in range(len(M1)):
26             M1[j] = RandomStep(M1[j], Lambda)
27
28         # elimination
29         M1 = g(M1)
30
31         # fission
32         for t in range(len(M1)):
33             if np.random.random() < k:
34                 M1.append(M1[t])
35
36     M1 = np.array(M1)
37     return M1
38
39 M2 = NeutronSimulate(50 , gcube , 0.4 , 0.3, 30)

```

1.4

In [5]:

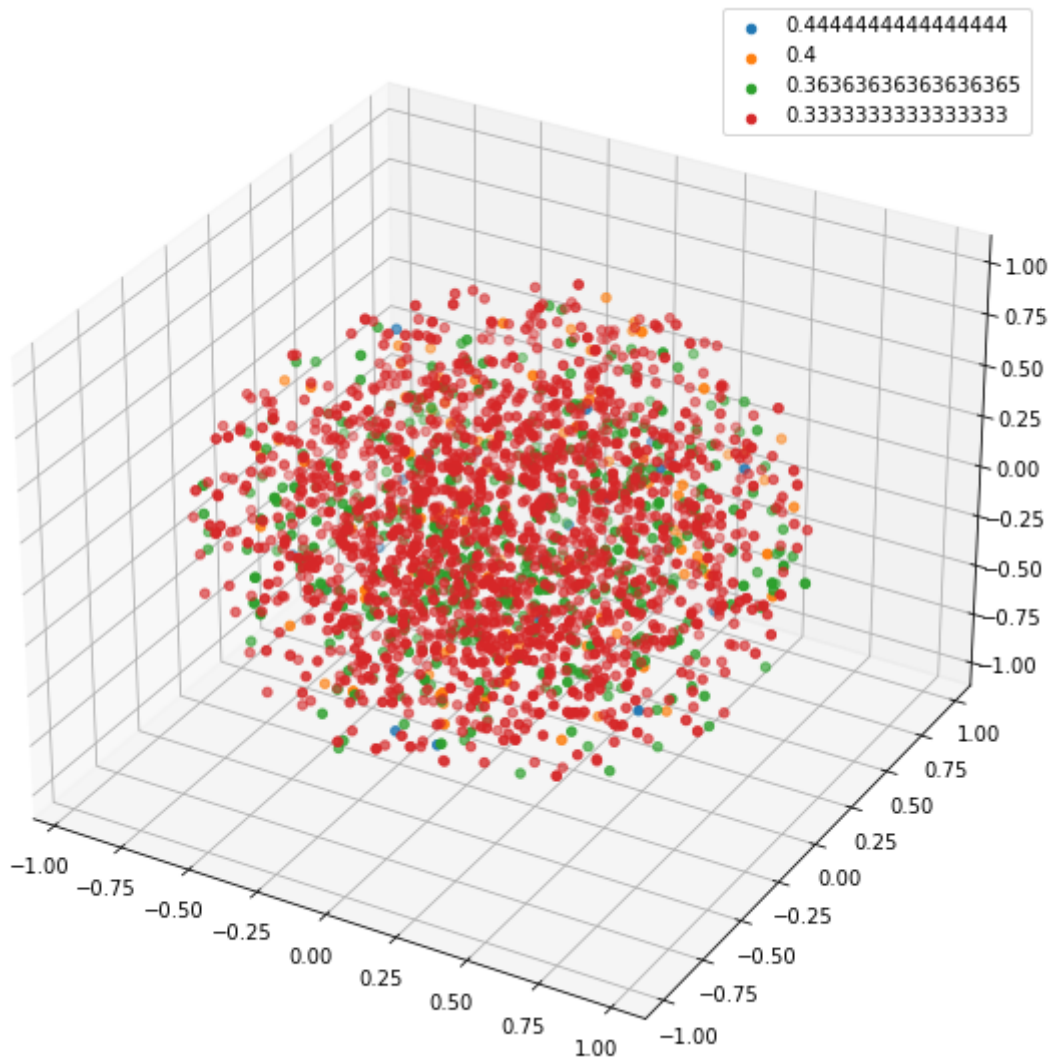
```
1 def gsphere(M):
2
3     """
4     receives Nx3 array of positions and returns N'x3
5     array of N' position that are inside g
6     """
7
8     MM = []
9     for i in range(len(M)):
10         if np.sqrt( (M[i][0])**2 + (M[i][1])**2 + (M[i][2])**2 ) < 1:
11             MM.append(M[i])
12
13     return MM
```

In [87]:

```

1 %matplotlib inline
2
3 fig = plt.figure(figsize=[10,10])
4 ax = fig.add_subplot(111, projection='3d')
5 A = [4/6 , 4/7, 4/8, 4/9, 4/10, 4/11, 4/12]
6 B = [4/9, 4/10, 4/11, 4/12]
7 for i in A:
8     P1 = NeutronSimulate(100 , gsphere , i , 0.3, 50)
9     if len(P1) != 0:
10         ax.scatter(P1[:,0],P1[:,1],P1[:,2], 'o-', label = i)
11         plt.legend()
12
13 plt.show()
14
15 # for i in A:
16 #     P2 = NeutronSimulate(100 , gcube , 0.2*i , 0.5, 10)
17 #     if len(P2) != 0:
18 #         ax.scatter(P2[:,0],P2[:,1],P2[:,2], 'o-', label = i)
19 #         plt.legend()
20
21 # plt.show()

```



In [53]:

```

1 uranium_density = 19 #g/cm^3
2 Rc = 9 #cm
3 Vc = (4/3)*np.pi*Rc**3 #cm^3
4 Mc = Vc*uranium_density #g
5 Mc_kg = Mc/1000
6 Mc_kg

```

Out[53]:

58.018933126496286

In [68]:

```

1 Vsphere = [[],[],[],[],[],[],[],[ ]
2 Vcube = [[],[],[],[],[],[],[],[ ]
3 N = 10
4 for i in range(len(A)):
5     for j in range(N):
6         Vsphere[i].append( len(NeutronSimulate(100 , gsphere , A[i] , 0.3, 50)) )
7         Vcube[i].append( len(NeutronSimulate(100 , gcube , A[i] , 0.3, 50)) )

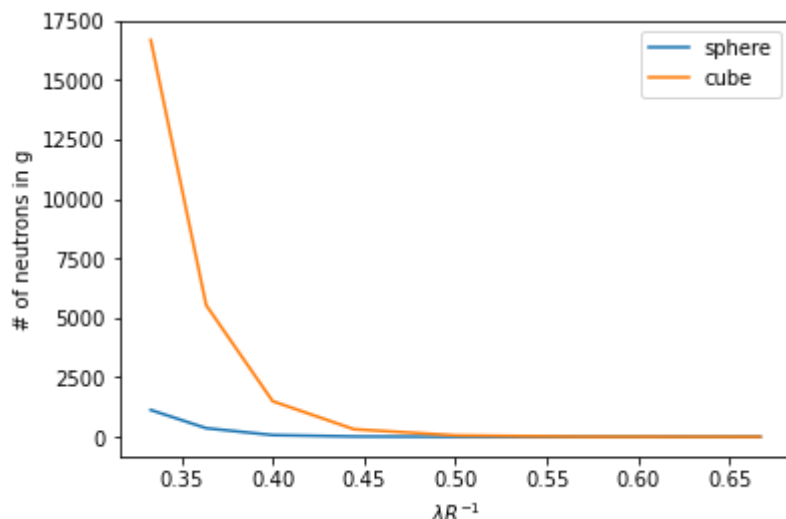
```

In [81]:

```

1 VsphereMean = []
2 VsphereStd = []
3 VcubeMean = []
4 VcubeStd = []
5 for i in range(7):
6     VsphereMean.append( np.mean(Vsphere[i]) )
7     VsphereStd.append( np.std(Vsphere[i]) )
8     VcubeMean.append( np.mean(Vcube[i]) )
9     VcubeStd.append( np.std(Vcube[i]) )
10
11 plt.plot(A , VsphereMean, label ="sphere")
12 plt.plot(A , VcubeMean, label ="cube")
13 plt.ylabel("# of neutrons in g")
14 plt.xlabel("$ \\lambda R^{-1} $" )
15 plt.legend()
16 plt.show()

```



It is apparent that the "explosion" of neutrons occurs at a larger scale for the cube than it is for the sphere

for the sphere it occurs around 0.40 cm^{-1} which is $R = 10\text{cm}$

for the cube it occurs at less than 0.35 cm^{-1} which is $R = 11.42\text{cm}$